

5. Multimodal Visual Understanding + Visual Line Following

1. Course Content

1. Learn to use the robot's visual understanding combined with the line-following autonomous driving function.
2. Analyze newly appearing key source code.

2. Preparation

2.1 Content Description

The virtual machine used by default with this product starts the Dify Docker environment automatically at boot, so no additional startup is usually needed.

⚠ The same test instructions will not produce exactly the same reply content from the large model, and will differ slightly from the screenshots in the tutorial. If you need to enhance or reduce the diversity of the large model's replies, refer to **【AI Large Model Development】 - 【RAG Knowledge Base Deployment】** to modify the knowledge base and sample library.

Visual line following depends on color HSV calibration. Before starting this case, it is recommended to complete the calibration of the four colors red, green, blue, and yellow according to the color calibration process in **【AI Large Model Preparation】**, and confirm that the `color_hsv.txt` file has been updated.

Virtual machine terminal, start the Dify environment:

```
sh ~/bringup_dify.sh
```

```
yahboom@yahboom:~$ sh ~/bringup_dify.sh
[+] up 13/13
✓ Container docker-sandbox-1      Running      0.0s
✓ Container docker-web-1          Running      0.0s
✓ Container docker-db_postgres-1  Healthy     22.5s
✓ Container docker-ssrf_proxy-1   Running      0.0s
✓ Container docker-redis-1        Running      0.0s
✓ Container docker-weaviate-1     Running      0.0s
✓ Container docker-worker-1       Running      0.0s
✓ Container docker-api_websocket-1 Running      0.0s
✓ Container docker-plugin_daemon-1 Running      0.0s
✓ Container docker-worker_beat-1  Running      0.0s
✓ Container docker-api-1          Running      0.0s
✓ Container docker-nginx-1        Running      0.0s
✓ Container docker-init_permissions-1 Exited       28.4s
yahboom@yahboom:~$
```

3. Running the Case

3.1 Starting the Program

Open the virtual machine terminal and enter the command to start the underlying proxy + camera topic:

```
ros2 launch microros_v2_bringup car_base.launch.py
```

```
yahboom@yahboom:~$ ros2 launch microros_v2_bringup car_base.launch.py
[INFO] [launch]: All log files can be found below /home/yahboom/.ros/log/2026-07-23-21-30-23-024975-yahboom-608172
[INFO] [launch]: Default logging verbosity is set to INFO
[INFO] [micro_ros_agent-1]: process started with pid [608193]
[INFO] [camera_driver-2]: process started with pid [608195]
[INFO] [robot_state_publisher-3]: process started with pid [608197]
[INFO] [joint_state_publisher-4]: process started with pid [608199]
[INFO] [ekf_node-5]: process started with pid [608201]
[micro_ros_agent-1] [1784813423.243996] info      | UDPv4AgentLinux.cpp | init
                        | running...          | port: 8090
[robot_state_publisher-3] [INFO] [1784813423.323961815] [robot_state_publisher]:
got segment base_footprint
[robot_state_publisher-3] [INFO] [1784813423.324159300] [robot_state_publisher]:
got segment base_link
[robot_state_publisher-3] [INFO] [1784813423.324177934] [robot_state_publisher]:
got segment bwheel_Link
[robot_state_publisher-3] [INFO] [1784813423.324185448] [robot_state_publisher]:
got segment cam1_Link
[robot_state_publisher-3] [INFO] [1784813423.324192058] [robot_state_publisher]:
got segment cam2_Link
[robot_state_publisher-3] [INFO] [1784813423.324198417] [robot_state_publisher]:
```

Open another virtual machine terminal and enter the command:

```
ros2 launch multi_brains llm_agent_control.launch.py
```

After initialization is complete, the following content will be displayed:

```
yahboom@VM: ~  
G_MODE=False, LANGUAGE=en, CAR_TYPE=ptz  
[llm_agent_node-1] [INFO] [1787128762.059479166] [llm_agent_node]: Initializing  
LLM Agent Node...  
[llm_agent_node-1] [INFO] [1787128762.070369334] [llm_agent_node]: [CONFIG] cmd_  
vel_topic=/cmd_vel  
[llm_agent_node-1] [INFO] [1787128762.077274270] [llm_agent_node]: [CONFIG] imag  
e_cache_path=/home/yahboom/microros_ws/cache/llm_camera.jpg  
[llm_agent_node-1] [INFO] [1787128762.078819208] [llm_agent_node]: [PTZ] startup  
initialization enabled: x=0.0°, y=-45.0°, count=20, interval=0.10s  
[llm_agent_node-1] [INFO] [1787128762.106861388] [llm_agent_node]: [FOLLOW] scan  
_topic=/scan, obstacle_guard=True, buzzer_topic=/beep, buzzer=True  
[llm_agent_node-1] [INFO] [1787128762.107460365] [llm_agent_node]: [DIFY] Initia  
lize client: base_url=http://localhost/v1, api_key=app-SAnX****Zt2E  
[llm_agent_node-1] [INFO] [1787128762.109741022] [llm_agent_node]: [DIFY] Testin  
g connection....  
[llm_agent_node-1] [INFO] [1787128762.130633809] [llm_agent_node]: [TRACK_KCF] S  
ubscribed /tracking_bbox, type=smart_tracker/msg/TrackBox  
[llm_agent_node-1] [INFO] [1787128762.131212772] [llm_agent_node]: LLM Agent Nod  
e initialized successfully  
[llm_agent_node-1] [INFO] [1787128762.133300019] [llm_agent_node]: [DIFY] Connec  
tion successful ✓  
[llm_agent_node-1] [INFO] [1787128764.080740503] [llm_agent_node]: [PTZ] Servo i  
nitialization complete: x=0.0°, y=-45.0°  
|
```

Then open another terminal and start the text dialogue:

```
ros2 run multi_brains terminal_client
```

```
yahboom@VM: ~  
> =====  
> MicroROS LLM Agent Terminal Client  
> =====  
> Enter natural-language commands to control the robot  
>  
> Examples:  
>   > Go to point A  
>   > Follow the red line  
>   > Look at what is ahead  
>   > Move forward 2 meters  
>  
> Commands:  
>   reset - Reset conversation context  
>   clear - Clear screen  
>   help  - Show this help  
>   exit  - Exit program  
> =====
```

3.2 Test Cases

Reference test cases are given here. Users can create their own dialogue commands:

- Start following the red line
- Start following the green line
- Start following the blue line
- Start following the yellow line

The colors include `red`, `green`, `blue`, and `yellow`. If the user does not specify a color, the execution layer will output `follow_line(red)` by default.

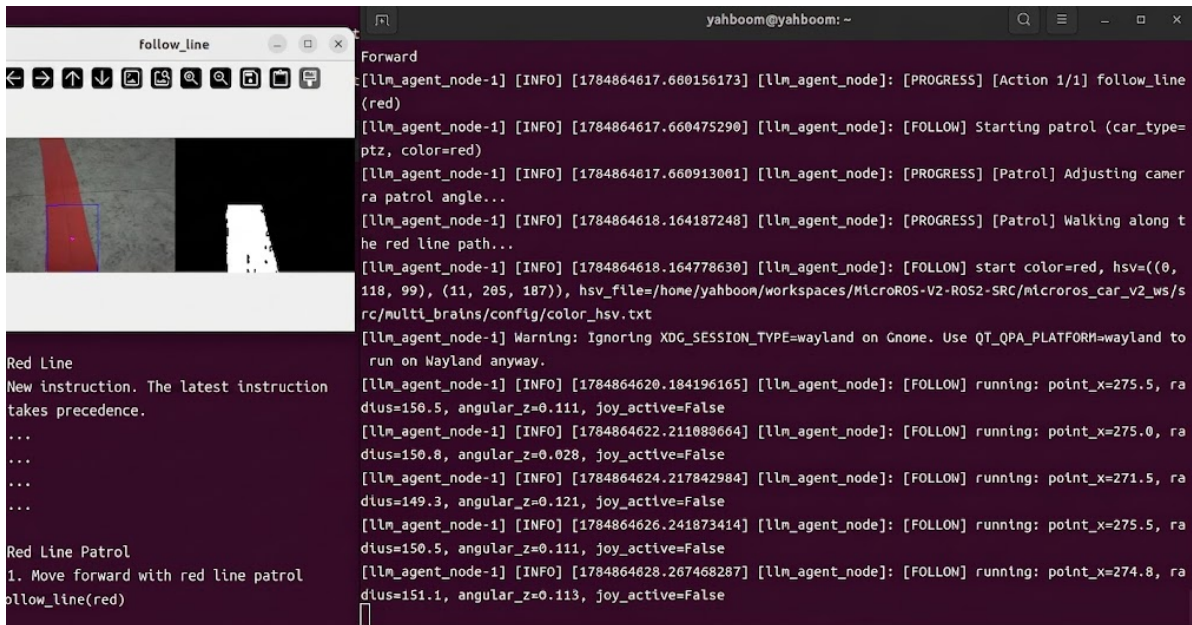
3.2.1 Case: "Start following the red line"

After the program starts, it will first adjust the camera to the line-following angle to look at the line on the ground.

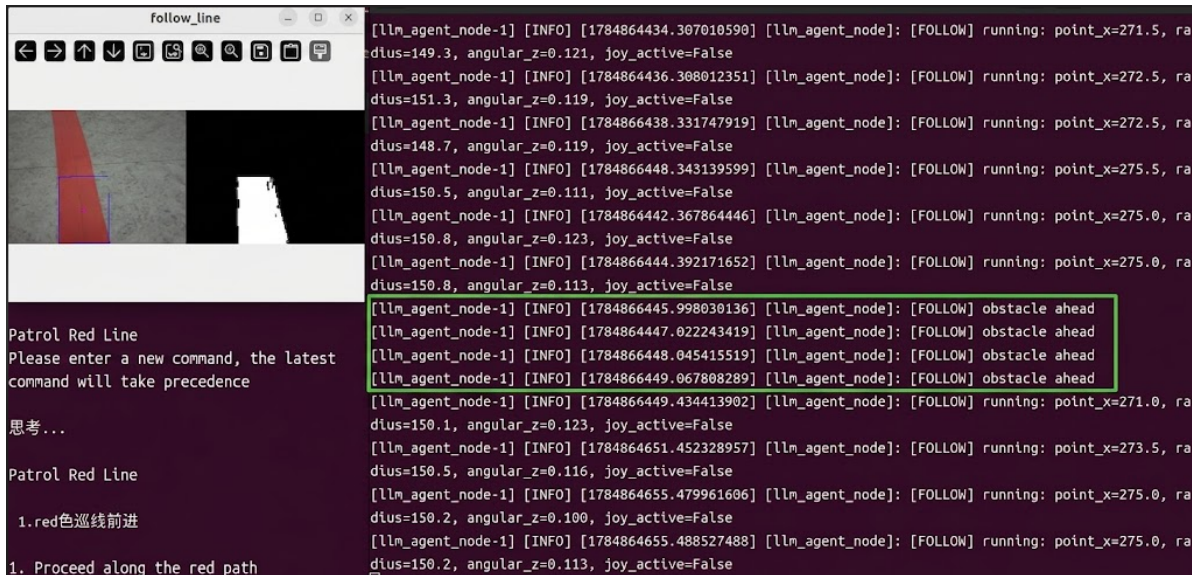
Enter "Start following the red line" in the text dialogue terminal and press Enter:

```
> ["observe_surroundings()", "finishtask()"]  
> -----  
> Red line patrol initiated
```

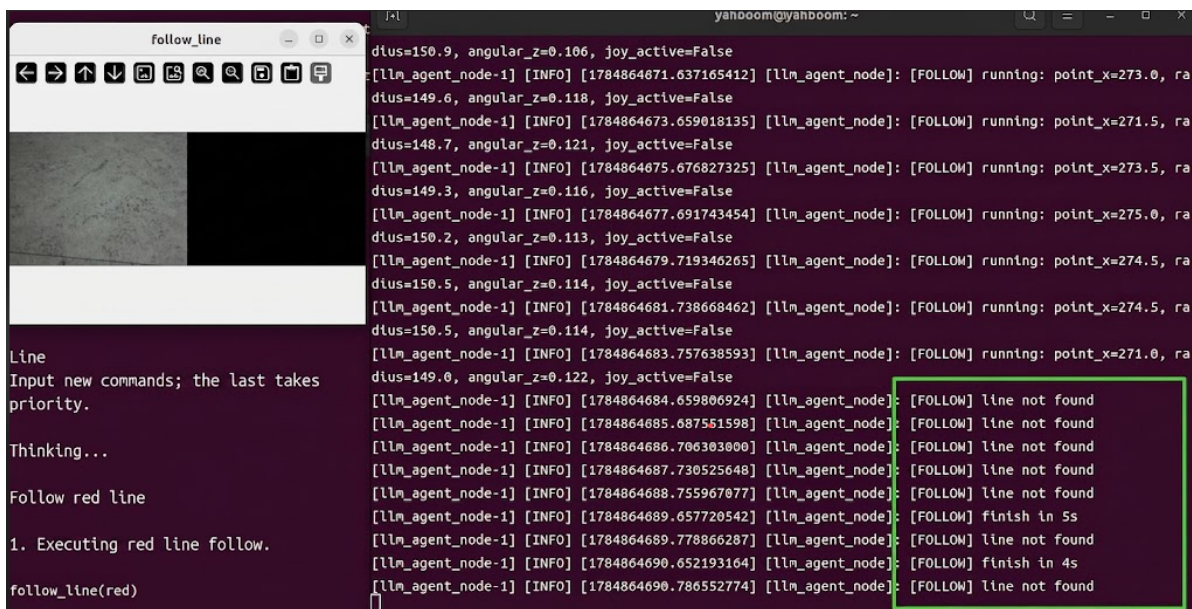
The execution layer will convert the text instruction into `follow_line(green)`. The robot will read the green HSV calibration values, recognize the green line in the lower half of the frame, and continuously publish chassis velocity to control the car to walk along the line.



During line following, if an obstacle is encountered, the car stops moving and the buzzer starts beeping. After the obstacle is removed, the program will continue trying to recognize the line and resume line following.



If the corresponding color line is not recognized in the frame for a long time, the program will count down 10 seconds and automatically stop, treating the current line-following task as ended.



To end the task manually, you can press the `ENTER` key in the terminal to continue the dialogue input:

- Stop line following
- Stop task
- Park

After the robot completes the task, it enters a waiting state. At this time, instructions will continue to be passed to the execution-layer large model, and the conversation history will be retained. You can input the `/reset` command again to let the robot end the current task cycle and start a new task cycle.

4. Source Code Analysis

In the current version, visual line following is implemented directly by `FollowLineController` inside the `multi_brains` package, and no longer starts the old-version line-following subprocess.

Current relevant source code paths:

```
~/microros_ws/src/multi_brains/multi_brains/llm_agent_node.py
~/microros_ws/src/multi_brains/multi_brains/controllers/follow_line_controller.py
~/microros_ws/src/multi_brains/config/llm_agent_config.yaml
~/microros_ws/src/multi_brains/config/color_hsv.txt
```

Execution layer actions:

```
follow_line(red)
follow_line(green)
follow_line(blue)
follow_line(yellow)
```

Core flow:

```
User inputs "start following the green line"
-> Execution layer outputs follow_line(green)
-> llm_agent_node adjusts the pan-tilt to the line-following angle
-> FollowLineController reads the HSV calibration values from color_hsv.txt
-> Detects the corresponding color line in the lower half of the frame based on
/camera/image_raw
-> Publishes /cmd_vel to control the chassis to walk along the line
-> Detects obstacles ahead via /scan, stops and turns on the buzzer if
necessary
-> Stops when the line is lost, a timeout occurs, an obstacle is detected, or a
new instruction is received
```

The first four non-empty lines of `color_hsv.txt` correspond to red, green, blue, and yellow in order. Parameters such as line-following speed, PID parameters, line-loss timeout, and obstacle distance threshold are located in the `FOLLOW_LINE` configuration section of `llm_agent_config.yaml`.

If the user does not specify a color, the execution layer outputs `follow_line(red)` by default. If you need to adjust the color recognition effect, first update `config/color_hsv.txt` according to the color calibration process in **【AI Large Model Preparation】**.

